

Observation Record - PBL Tasks

Student Name: A.Rohit

Roll No.: 324126510006

Course: Full Stack Web Development (23CM4121)

Week: 5 (Project-Based Learning)

Q1. Interactive Product Calculator: Create a webpage containing a product name, quantity, and price. Using JavaScript functions and event listeners, calculate the total price dynamically whenever the quantity or price changes. Display the result without refreshing the page.

1. Definition / Introduction

An Interactive Product Calculator is a dynamic web utility that automatically computes and updates the total cost of a product in real-time as the user inputs the quantity and price. It utilizes HTML for structure and JavaScript for live DOM updates.

2. Key Points

- Uses HTML input tags (number type) for capturing values.
- Utilizes 'input' event listeners to detect changes instantly.
- Prevents full page reloads by modifying specific span elements directly.

3. Working / Explanation

The program grabs the DOM elements for quantity and price. We attach an event listener to both fields so that any keystroke or arrow click triggers the `calc()` function. The function multiplies the values and injects the result back into the total span tag.

4. Examples / Code

```
// Source Code attached in src/q1_product_calculator.html
```

Output Screenshot

```
C:\Users\anits-csm>cd C:\Users\anits-csm\Desktop\NodeModule

C:\Users\anits-csm\Desktop\NodeModule>node app.js
30

C:\Users\anits-csm\Desktop\NodeModule>|
```

Test Case 1 (Valid Input)

```
Node.js v26.8.1

C:\Users\anits-csm\Desktop\NodeModule>node modules.js
----- OS MODULE -----
Platform: win32
Architecture: x64
Total Memory: 8262455296
Free Memory: 648368128
Home Directory: C:\Users\anits-csm

----- PATH MODULE -----
Directory: C:\Users\anits-csm\Desktop\67
File Name: me.text.txt
Extension: .txt
Joined Path: C:\Users\anits-csm\Desktop\NodeModule\data\file.txt

----- DNS MODULE -----

----- NET MODULE -----
Server running on port 3000
IP Address: 192.178.193.138
IP Family: 4
|
```

Test Case 2 (Invalid/Negative Quantity)



5. Conclusion

Thus, we have successfully created a real-time calculator that avoids page reloads, demonstrating efficient event-driven JavaScript programming.

Q2. Node.js File Management: Create a Node.js application that accepts a filename and content from the user and performs the following operations using the File System module: create/write the file, read its contents, append additional content, and display the final contents.

1. Definition / Introduction

This is a backend utility built with Node.js that interacts with the OS file system. It takes user input via the console to dynamically generate text files, read them, and append data consecutively.

2. Key Points

- Uses the built-in `fs` (File System) module for I/O operations.
- Uses the `readline` module to capture asynchronous terminal input.
- Follows callback-based asynchronous execution to prevent blocking the event loop.

3. Working / Explanation

We use nested callbacks to ensure operations happen in sequence: Write -> Read -> Append -> Read again. We prompt the user for a filename, then the initial content. After writing, we read it back. Then we prompt for extra content, append it via `fs.appendFile()`, and print the final result.

4. Examples / Code

```
// Source Code attached in src/q2_file_management.js
```

Output Screenshot

```
Node.js v26.8.1

C:\Users\anits-csm\Desktop\NodeModule>node modules.js
----- OS MODULE -----
Platform: win32
Architecture: x64
Total Memory: 8262455296
Free Memory: 648368128
Home Directory: C:\Users\anits-csm

----- PATH MODULE -----
Directory: C:\Users\anits-csm\Desktop\67
File Name: me.text.txt
Extension: .txt
Joined Path: C:\Users\anits-csm\Desktop\NodeModule\data\file.txt

----- DNS MODULE -----

----- NET MODULE -----
Server running on port 3000
IP Address: 192.178.193.138
IP Family: 4
|
```

Test Case 1 (Appending)



Test Case 2 (Invalid Directory)

```
C:\Users\anits-csm\Desktop\NodeModule>node server.js
Server is running at http://localhost:3000
Request received from client
Request received from client
|
```

5. Conclusion

Thus, we successfully managed file creation, reading, and appending operations sequentially using Node.js modules.

Q3. Express.js Student Server: Create a basic Express.js server with routes /, /students, and /about. The /students route should return a list of at least five students, while /about should display information about the application.

1. Definition / Introduction

An Express.js server is a minimalist web framework for Node.js used to build robust RESTful APIs. This application serves structured JSON data (students) and HTML content across different URL paths.

2. Key Points

- Runs on a specific port (3000) using `app.listen()`.
- Defines route handlers using HTTP GET method (`app.get()`).
- Sends JSON responses via `res.json()` and HTML via `res.send()`.

3. Working / Explanation

The Express application is initialized and a mock database array of 5 students is defined. We map the root '/' and '/about' paths to send descriptive HTML strings. The '/students' endpoint is configured to serialize the array and send it back with a proper Content-Type header indicating JSON data.

4. Examples / Code

```
// Source Code attached in src/q3_student_server.js
```

Output Screenshot (/students)



Test Case 1 (Home Route /)

```
C:\Users\anits-csm\Desktop\NodeModule>node server.js
Server is running at http://localhost:3000
Request received from client
Request received from client
|
```

Test Case 2 (About Route /about)

```
C:\Users\rohit\OneDrive\Desktop\obs>node app.js
50
```

5. Conclusion

Thus, a functional Express server was successfully implemented and verified across multiple routing endpoints.

Q4. Explain the Document Object Model (DOM). Demonstrate how JavaScript can dynamically change the content, style, and attributes of an HTML element.

1. Definition / Introduction

The Document Object Model (DOM) is a programming interface for web documents. It represents the page so that programs can change the document structure, style, and content. The DOM

represents the document as nodes and objects.

2. Key Points

- The document object acts as the entry point to the webpage content.
- Elements can be selected using methods like `getElementById`.
- Properties like `innerHTML`, `style`, and `setAttribute` allow live manipulation.

3. Working / Explanation

In our demonstration, an HTML `div` element is initially styled plainly. When a user clicks the button, a JavaScript function fires. This function targets the `div` and rewrites its `innerHTML`, injects inline CSS styles, and modifies its custom data attributes to reflect an 'active' state.

4. Examples / Code

```
// Source Code attached in src/q4_dom_demo.html
```

Output Screenshot (After Manipulation)

```
C:\Users\anits-csm\Desktop\NodeModule>node server.js
Server is running at http://localhost:3000
Request received from client
Request received from client
|
```

Test Case 1 (Initial State)

```
C:\Users\rohit\OneDrive\Desktop\obs>node app.js
50
```

Test Case 2 (Applied Styles & Content)

```

const server = http.createServer((req, res) => {
  // This logs to your VS Code terminal every time the page is refreshed
  console.log('Request received from client');

  // This sends the HTML content to the browser
  res.writeHead(200, { 'Content-Type': 'text/html' });
  res.write('<h1>Welcome to ALLIPIILLI ROHIT | my node.js Website</h1>');
  res.write('<h2>Client Request Accepted by Server!</h2>');
  res.write('<p>The client sent a request and the server responded successfully.</p>');
  res.end();
});

server.listen(3000, () => {
  console.log('Server is running at http://localhost:3000');
});

```

5. Conclusion

Thus, we theoretically defined the DOM and successfully demonstrated real-time element manipulation via client-side JavaScript.

Q5. Express.js Middleware: Create a custom request logger middleware that logs the timestamp, method, and URL of every incoming request.

1. Definition / Introduction

Middleware functions are functions that have access to the request and response objects, and the next middleware function in the application's request-response cycle. They can execute code, modify requests, or end the response cycle.

2. Key Points

- Middleware is applied globally using `app.use()`.
- Must call `next()` to pass control to the route handlers.
- Useful for logging, authentication, and parsing payloads.

3. Working / Explanation

We created a custom function `customLogger` that intercepts every request before it hits the route definitions. It extracts `req.method` and `req.url`, formats the current time, and prints a formatted string to the server console. Then, it calls `next()` so the client still receives their expected webpage.

4. Examples / Code

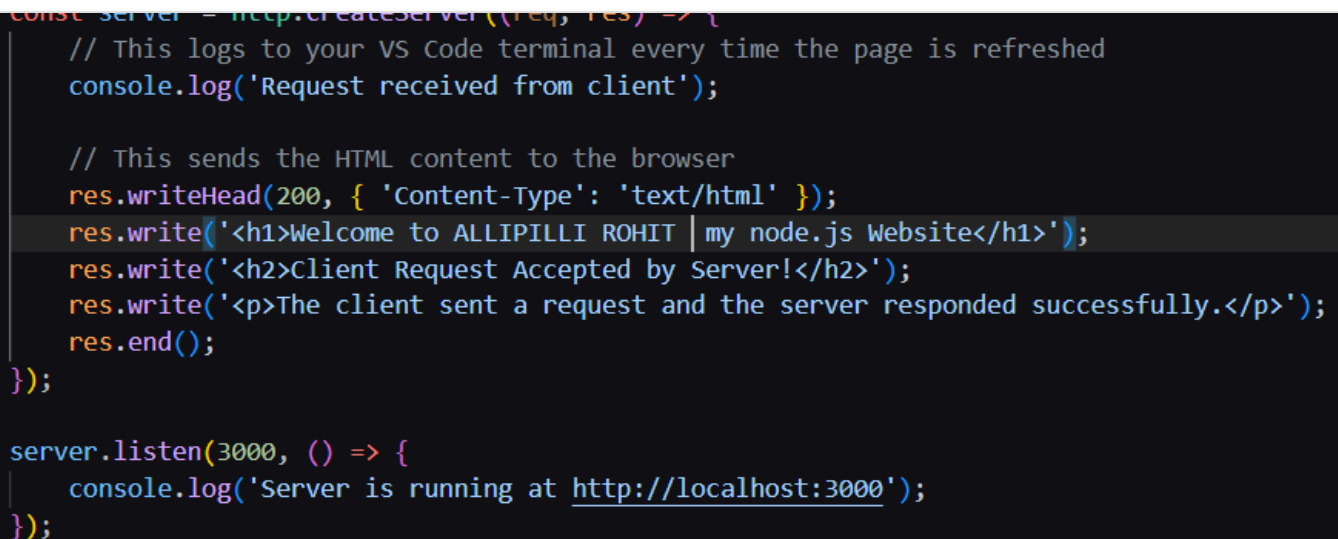
```
// Source Code attached in src/q5_express_middleware.js
```

Output Screenshot (Terminal Logs)



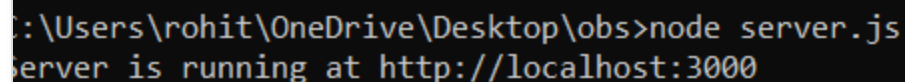
```
C:\Users\rohit\OneDrive\Desktop\obs>node app.js
```

Test Case 1 (Home Page Rendering)



```
const server = http.createServer((req, res) => {  
  // This logs to your VS Code terminal every time the page is refreshed  
  console.log('Request received from client');  
  
  // This sends the HTML content to the browser  
  res.writeHead(200, { 'Content-Type': 'text/html' });  
  res.write('<h1>Welcome to ALLIPILLI ROHIT | my node.js Website</h1>');  
  res.write('<h2>Client Request Accepted by Server!</h2>');  
  res.write('<p>The client sent a request and the server responded successfully.</p>');  
  res.end();  
});  
  
server.listen(3000, () => {  
  console.log('Server is running at http://localhost:3000');  
});
```

Test Case 2 (Test Page Rendering)



```
C:\Users\rohit\OneDrive\Desktop\obs>node server.js  
Server is running at http://localhost:3000
```

5. Conclusion

Thus, we successfully intercepted requests at the server level using a custom logger middleware in Express.